**UNIVERSIDAD PANAMERICANA**

Ingeniería y ciencias aplicadas

Ingeniería en sistemas

Programación V – Base de datos II

# Hipstagram

(Base de datos — Rol DBA)

Kevin Eliazar Herrera Alburez (000113637)

Carlos Enrique Otoniel Bautista Sarazua (000133047)

Herbert David Estrada Trujillo (00141126)

Chimaltenango, Chimaltenango, abril 2026

**Hipstagram**

(Base de datos — Rol DBA)

Kevin Eliazar Herrera Alburez (000113637)

Carlos Enrique Otoniel Bautista Sarazua (000133047)

Herbert David Estrada Trujillo (000141126)

Mario Obed Morales Güitz (Docente)

Chimaltenango, Chimaltenango, abril 2026

1 Introducción 1

2 Configuración de Roles y privilegios (GRANT/REVOKE) 1

2.1 Definición de roles 1

2.2 Figura 1. 1

2.3 Figura 2. Gestión de roles creados 2

3 Integri-dad referencial (Foreign Keys) 2

3.1 Figura 3. 2

3.2 Figura 4. Error (SQLSTATE 23503) 3

4 Diccionario de datos 4

4.1 Tabla usuario 4

4.2 Tabla publicación 4

4.3 Tabla bitácora (auditoria) 5

4.4 Tabla interacción (Likes y votos) 5

4.5 Tabla comentario 5

# 1 Introducción

En el marco del desarrollo de Hipstagram, la gestión de la base de datos (DBA) es fundamental para garantizar la integridad, disponibilidad y seguridad de la información, la implementación de roles de acceso, restricciones de integridad referencial y las políticas de control de usuario en el motor PostgreSQL, cada componente del sistema debe interactuar con los datos bajo el principio de mínimo privilegio, mitigando riesgos de seguridad y errores operativos.

# 2 Configuración de Roles y privilegios (GRANT/REVOKE)

## 2.1 Definición de roles

| Rol | Descripción | Restricción |
|---|---|---|
| dba_role | Accesos totales para mantenimiento. | Ninguna. |
| App_role | Utilizado por el Backend para operaciones de negocio. | No tiene permisos de lectura sobre la tabla bitácora |
| Admin_read_role | Para auditoría y visualización de logs. | Solo puede realizar SELECT en la tabla bitácora. |

## 2.2 Figura 1.

```
//Creación de roles
CREATE ROLE dba_role WITH LOGIN PASSWORD '********';
CREATE ROLE app_role WITH LOGIN PASSWORD '********';
CREATE ROLE admin_read_role WITH LOGIN PASSWORD '********';

//Restricción específica de seguridad
// app_role no debe leer audit_logs(en tu caso la tabla se llama 'bitacora')
REVOKE ALL ON TABLE public.bitacora FROM app_role;

//admin_read_role solo SELECT en bitacora
GRANT SELECT ON TABLE public.bitacora TO admin_read_role;
```

## 2.3 Figura 2. Gestión de roles creados

Roles creados y privilegios asignado en el motor.

```
-- Table: public.publicaciones

-- DROP TABLE IF EXISTS public.publicaciones;

CREATE TABLE IF NOT EXISTS public.publicaciones
(
    id integer NOT NULL DEFAULT nextval('publicaciones_id_seq'::regclass),
    usuario character varying(100) COLLATE pg_catalog."default",
    descripcion character varying(128) COLLATE pg_catalog."default",
    hashtags text COLLATE pg_catalog."default",
    fecha timestamp without time zone DEFAULT CURRENT_TIMESTAMP,
    oculto boolean DEFAULT false,
    CONSTRAINT publicaciones_pkey PRIMARY KEY (id)
)

TABLESPACE pg_default;

ALTER TABLE IF EXISTS public.publicaciones
    OWNER to admin;
```

# 3 Integri-dad referencial (Foreign Keys)

Se ha configurado la restricción ON DELETE RETRICT en las tablas críticas. Esta regla impide eliminar registros padres si existen registros hijos vinculados, forzando la consistencia del sistema.

| Relación crítica | Tabla padre | Tabla hijo | Acción de borrado |
|---|---|---|---|
| Usuarios a Posts | Usuario | Publicación | RESTRICT |
| Posts a comentarios | Publicación | Comentario | RESTRICT |
| Posts a interacciones | Publicación | Interacción | RESTRICT |

## 3.1 Figura 3.

```
ALTER TABLE public.publicacion
ADD CONSTRAINT fk_usuario_publicacion
FOREIGN KEY (id_usuario) REFERENCES public.usuario(id)
ON DELETE RESTRICT;
```

## 3.2 Figura 4. Error (SQLSTATE 23503)

Al eliminar un registro protegido, el motor responde con un error de violación de restricción.

```
-- Table: public.usuarios

-- DROP TABLE IF EXISTS public.usuarios;

CREATE TABLE IF NOT EXISTS public.usuarios
(
    id integer NOT NULL DEFAULT nextval('usuarios_id_seq'::regclass),
    usuario character varying(50) COLLATE pg_catalog."default" NOT NULL,
    password character varying(255) COLLATE pg_catalog."default" NOT NULL,
    correo character varying(100) COLLATE pg_catalog."default",
    fecha_creacion timestamp without time zone DEFAULT CURRENT_TIMESTAMP,
    rol character varying(20) COLLATE pg_catalog."default" DEFAULT 'usuario'::character varying,
    activo boolean DEFAULT true,
    CONSTRAINT usuarios_pkey PRIMARY KEY (id),
    CONSTRAINT usuarios_correo_key UNIQUE (correo),
    CONSTRAINT usuarios_usuario_key UNIQUE (usuario)
)

TABLESPACE pg_default;

ALTER TABLE IF EXISTS public.usuarios
    OWNER to admin;
```

Para proteger la base de datos de borrados accidentales que dejen datos huérfanos, se configuraron llaves foráneas con la regla RESTRICT.

- **Usuario → Publicación:** Un usuario no puede ser eliminado si posee publicaciones.
- **Publicación → Comentario:** Una publicación no puede eliminarse si tiene comentarios vinculados.

La implementación de ON DELETE RESTRICT garantiza que la ampliación no sufra de inconsistencias lógicas, por ejemplo, al intentar eliminar un usuario que ya tiene registros en una tabla, el motor PostgreSQL aborta la operación inmediatamente, protegiendo la integridad de los datos históricos y evitando registros huérfanos en el sistema.

# 4 Diccionario de datos

Las tablas que componen la base de datos de Hipstagram, estos tipos de datos han sido seleccionados para optimizar y garantizar la integridad de la información.

## 4.1 Tabla usuario

Almacena la información de perfil y credenciales de acceso.

| Campo | Tipo de dato | Restricción | Descripción |
|---|---|---|---|
| Id | SERIAL | PRIMARY KEY | Identificador único del usuario. |
| Nombre_usuario | VARCHAR(100) | NOT NULL, UNIQUE | Nombre de visualización en la red. |
| correo | VARCHAR (150) | NOT NULL, UNIQUE | Correo electrónico para login. |
| password_hash | TEXT | NOT NULL | Contraseña cifrada. |
| biografia | TEXT | NULL | Breve descripción del perfil. |
| foto_perfil_url | TEXT | NULL | Link al bucket de S3. |

## 4.2 Tabla publicación

Contiene la metadata de las fotos subidas por los usuarios.

| Campo | Tipo de dato | Restricción | Descripción |
|---|---|---|---|
| id | SERIAL | PRIMARY KEY | ID de la publicación. |
| id_usuario | INTEGER | FK (usuario.id) | Referencia al autor (RESTRICT). |
| correo | VARCHAR (150) | NOT NULL, UNIQUE | Correo electrónico para login. |
| url_foto | TEXT | NOT NULL | URI del recurso en Amazon S3. |
| descripcion | TEXT | NULL | Pie de foto. |
| Fecha_creacion | TIMSTAMP | DEFAULT NOW() | Fecha y hora de subida. |

## 4.3 Tabla bitácora (auditoria)

Registro de eventos críticos para el rol DBA y auditoria.

| Campo | Tipo de dato | Restricción | Descripción |
|---|---|---|---|
| id | SERIAL | PRIMARY KEY | ID del registro de bitácora. |
| usuario | VARCHAR(100) | NOT NULL | Usuario que realizó la acción. |
| accion | TEXT | NOT NULL | Descripción de la operación. |
| fecha | TIMESTAMP | DEFAULT NOW() | Timestamp del evento. |

## 4.4 Tabla interacción (Likes y votos)

| Campo | Tipo de dato | Restricción | Descripción |
|---|---|---|---|
| id | SERIAL | PRIMARY KEY | ID de la interacción. |
| id_usuario | INTEGER | FK (usuario.id) | Usuario que reacciona. |
| id_publicacion | INTEGER | FK (publicacion.id) | Post reaccionado. |
| tipo | VARCHAR(50) | NOT NULL | Tipo de reacción (Like/Voto). |

## 4.5 Tabla comentario

| Campo | Tipo de dato | Restricción | Descripción |
|---|---|---|---|
| id | SERIAL | PRIMARY KEY | ID del comentario. |
| id_usuario | INTEGER | FK (usuario.id) | Autor del comentario. |
| id_publicacion | INTEGER | FK (publicacion.id) | Post comentado. |
| contenido | TEXT | NOT NULL | Cuerpo del mensaje. |